Pipeline-Aware Offload

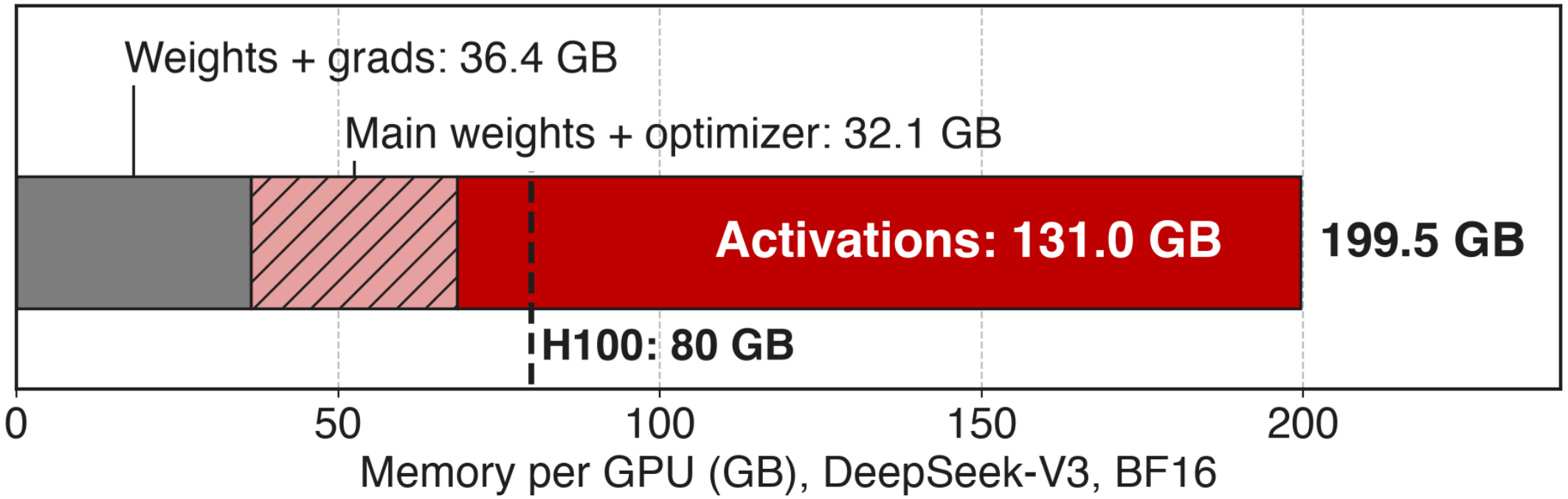
Activations and optimizer state to host memory, on the pipeline clock

NVIDIA Megatron-Core · Moonshot Kimi K2 / K3

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Memory & Offload Scheduling · arxiv.org/abs/2603.07685

Activations and optimizer state overflow HBM



DeepSeek-V3 (685B total, 37B active), BF16, PP4 × VPP4 × EP64 on 256 GPUs (Megatron-Core Table 3)

Meanwhile Kimi K2's H800 nodes carry 2 TB of host RAM per 8 GPUs

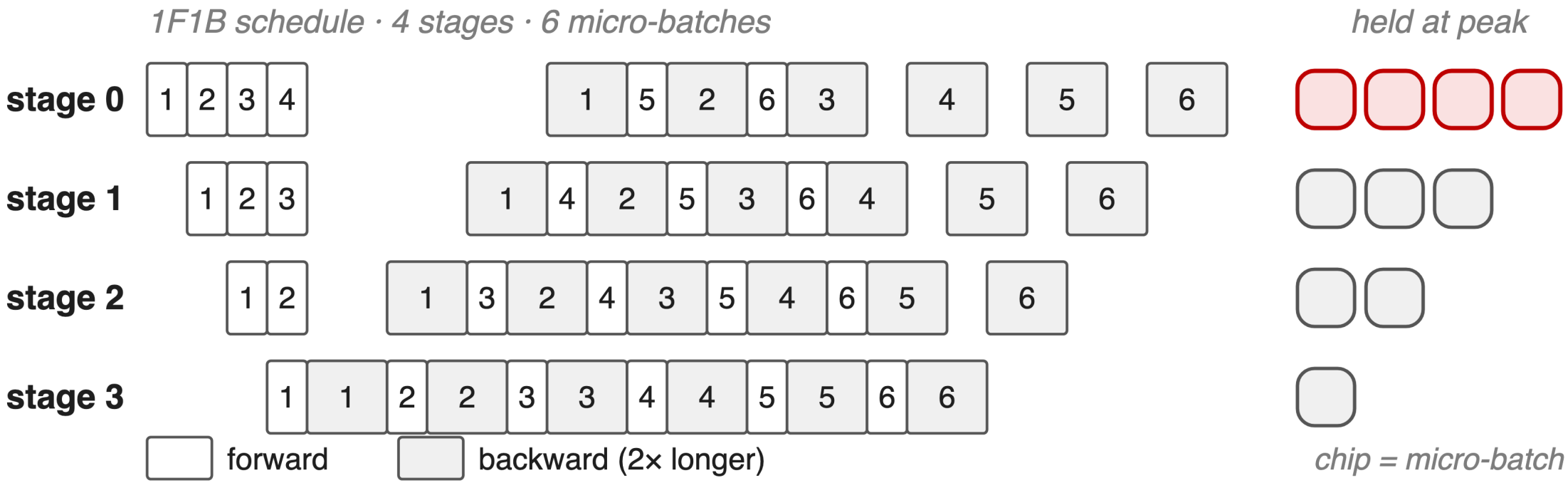
Memory, not speed, picks the parallel layout

15%

Qwen3-235B throughput forgone while memory pins it at TP2 / EP16 (800 TF/s) instead of TP1 / EP64 (920 TF/s)

Megatron-Core MoE technical report, arXiv 2603.07685, §4.1.5 and Table 5 (p.25)

Activations pile up on the first pipeline stage



EP and PP cut parameters per GPU, not this activation pile

Full recompute and layer offload both fall short

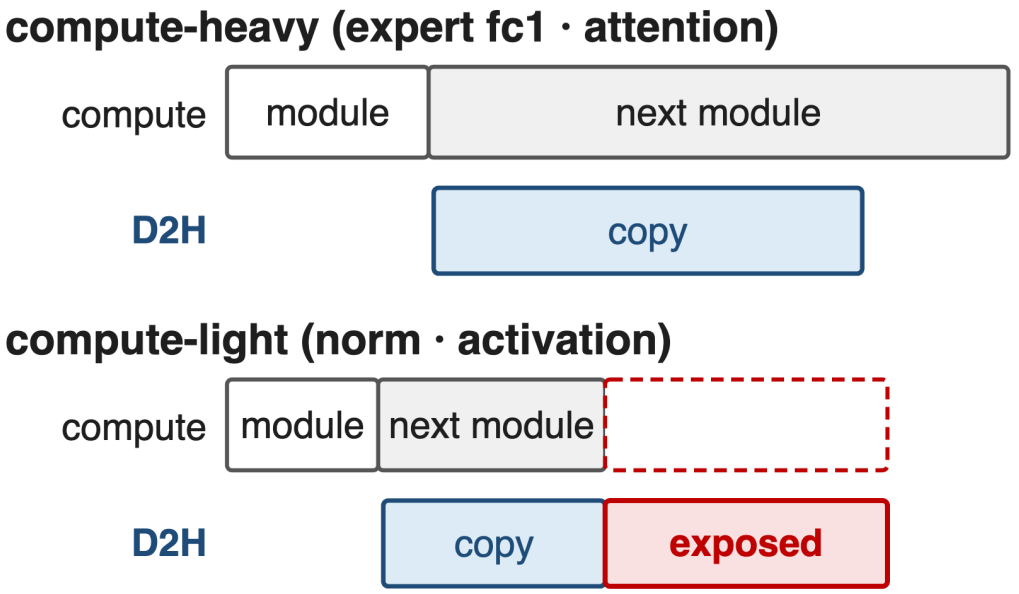
- **Full recompute**
- Keeps every layer's input: **L × layer input** on GPU
- ~33% more compute, replays all-to-all (deck 52)

- **Layer-level offload** (TE)
- **All or nothing** per layer
- Legacy path rejects PP > 1 and recompute

Needed: per-module choice, timed by the pipeline schedule

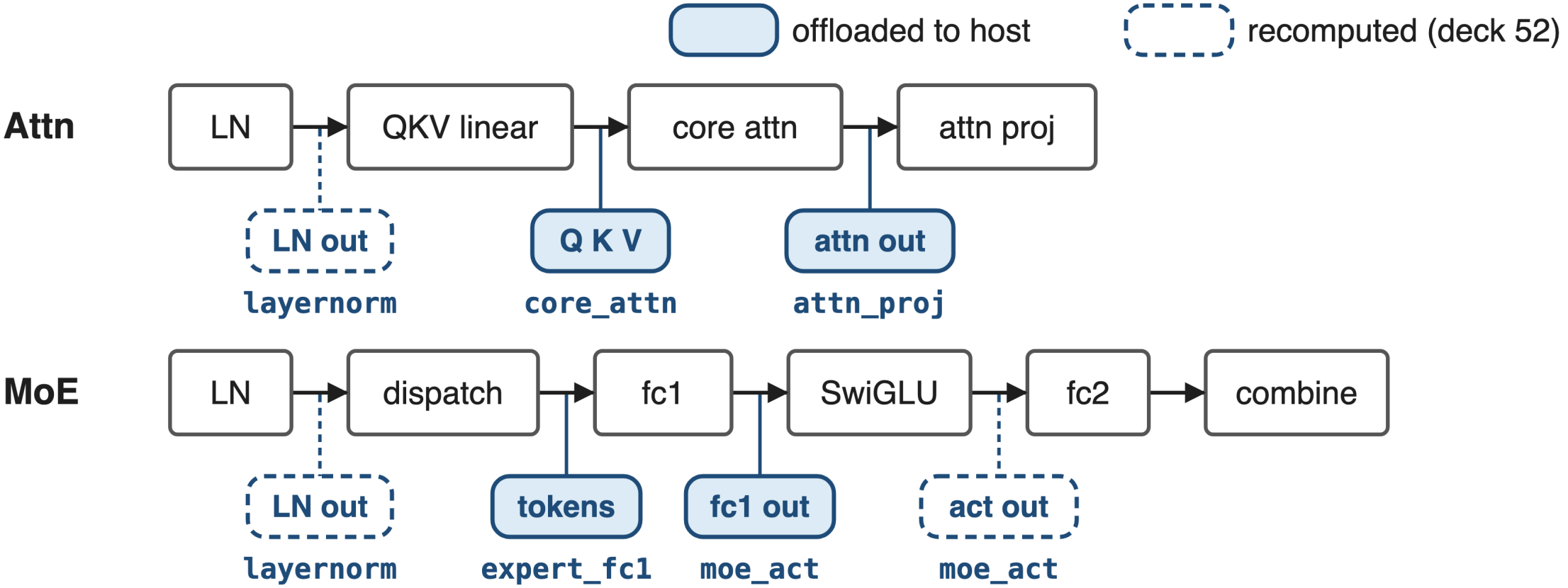
Insight: copies are free while a module computes

- Copy engine runs beside the SMs: hidden if compute outlasts the copy
- 1F1B fixes when each activation returns



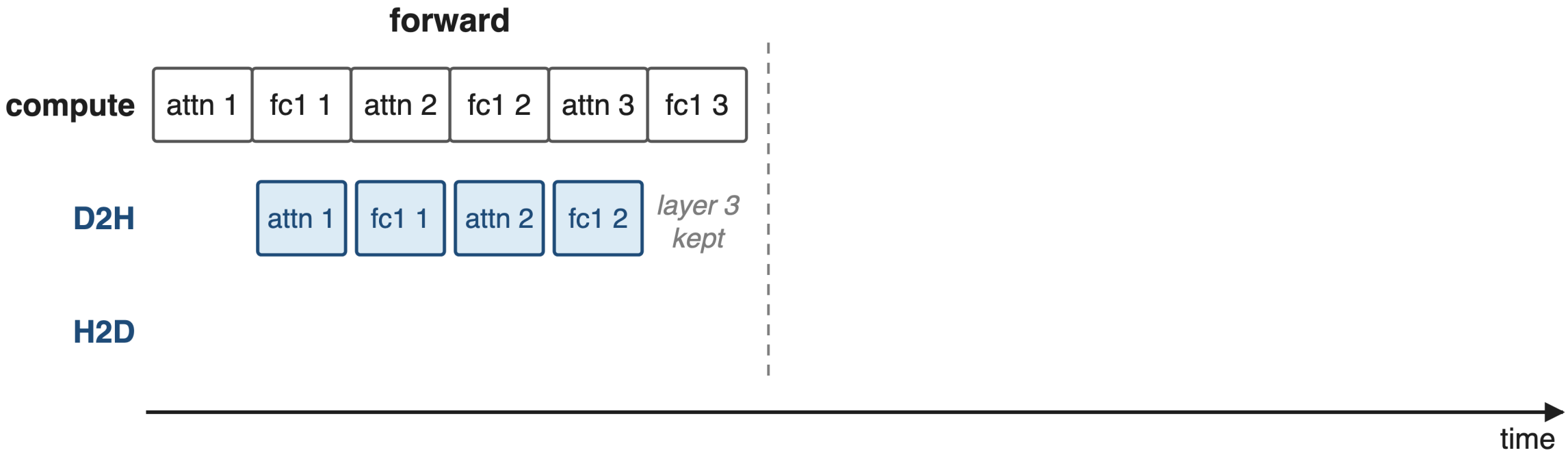
Offload heavy modules' inputs after forward; reload just before backward

Offload heavy inputs, recompute cheap outputs

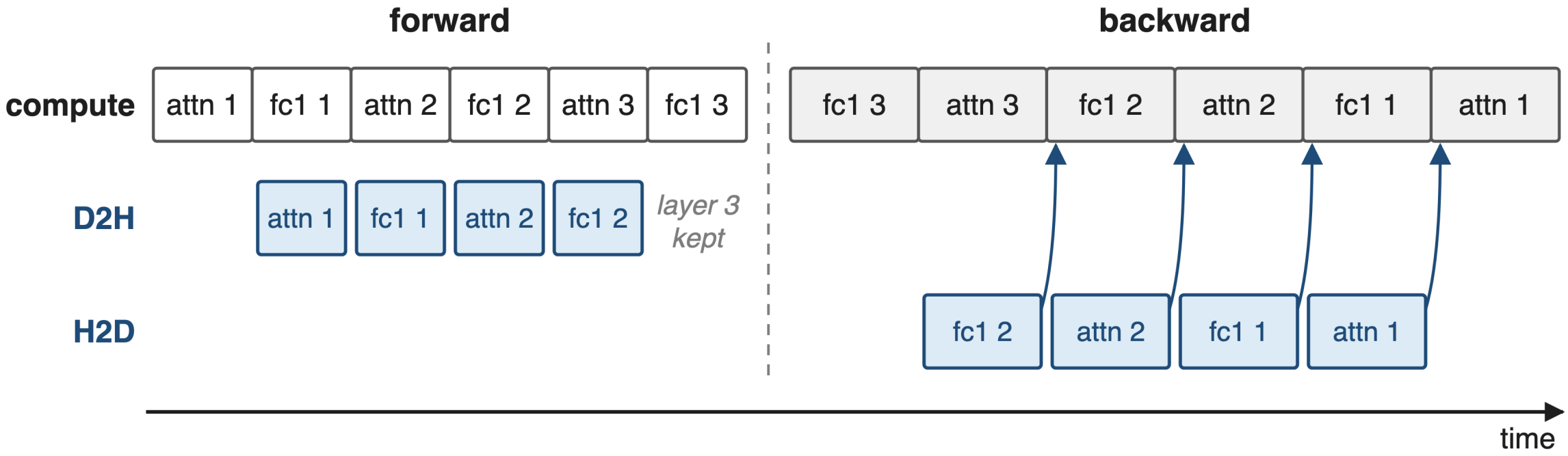


--offload-modules core_attn attn_proj expert_fc1 moe_act with --recompute-modules layernorm moe_act

❶ Copy out after forward, back one layer early

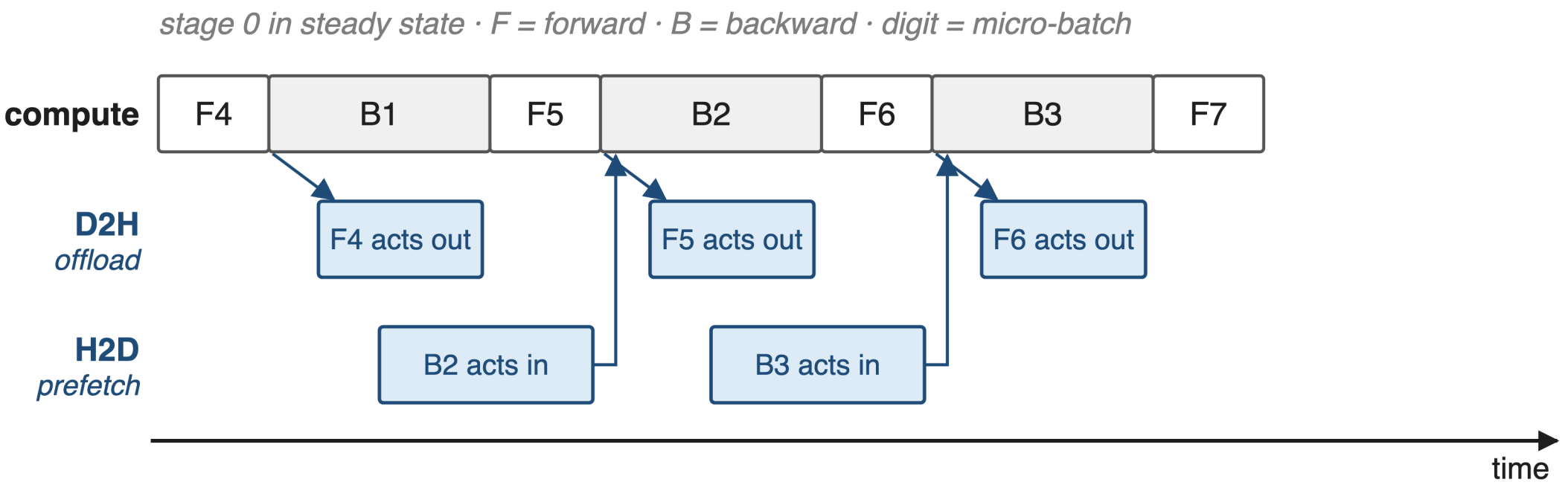


❶ Copy out after forward, back one layer early



The last layer stays on GPU; one reload per module type is in flight

❷ The 1F1B schedule decides what moves when



One handler per micro-batch × VPP chunk; later PP ranks offload less (`--delta-offload-bytes-across-pp-ranks`)

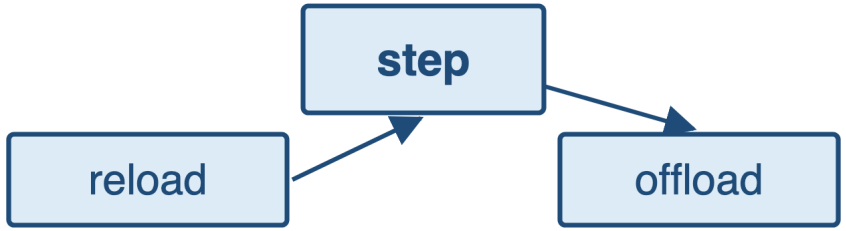
Kimi K2: offload the previous forward, prefetch the next backward

❸ Optimizer state leaves HBM between steps

state offload
step on GPU

forward + backward of all micro-batches

HBM free of state



CPU optimizer
step on host

forward + backward of all micro-batches

host CPU does the math



host side

state = exp_avg + exp_avg_sq + FP32 master weights

`--offload-optimizer-states` *(dev branch; now* `--chunked-optimizer-state-offload`*) vs* `--optimizer-cpu-offload`

Offload costs 1.6% speed, or buys back 15%

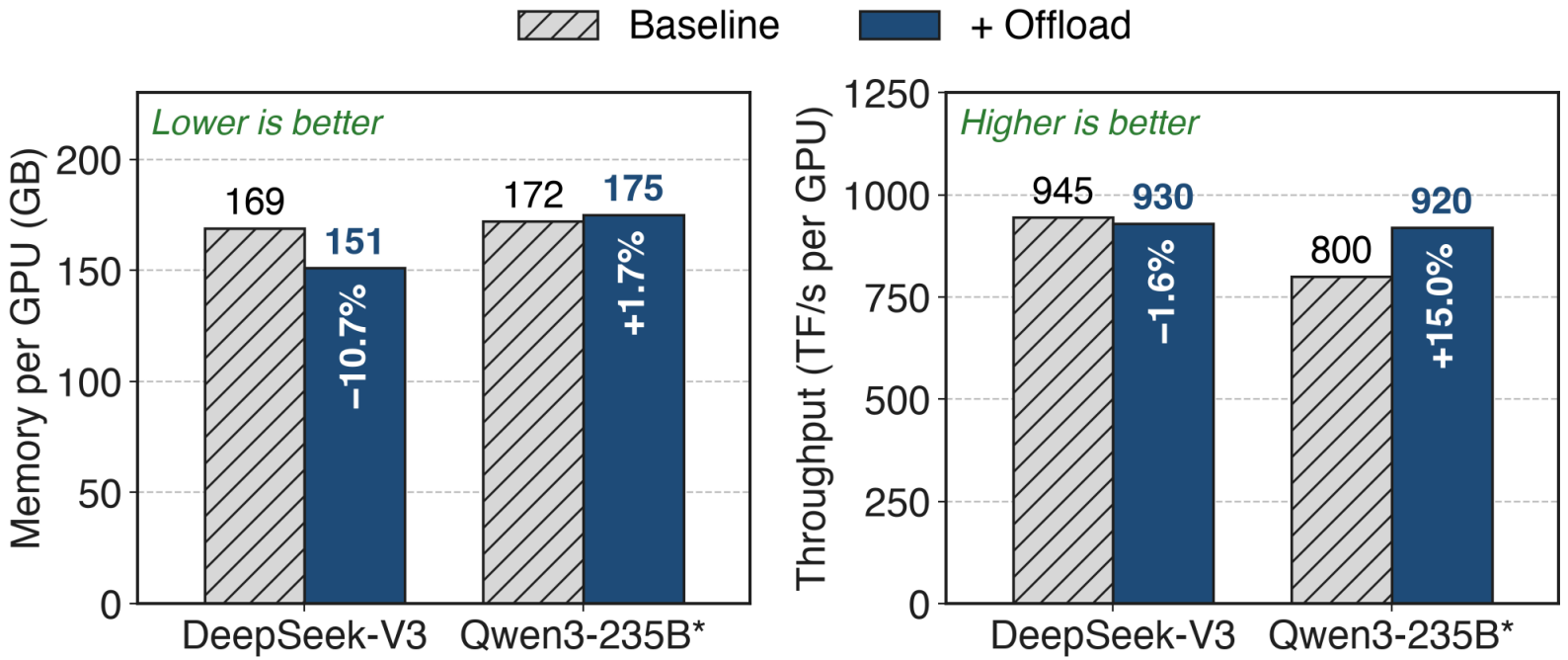
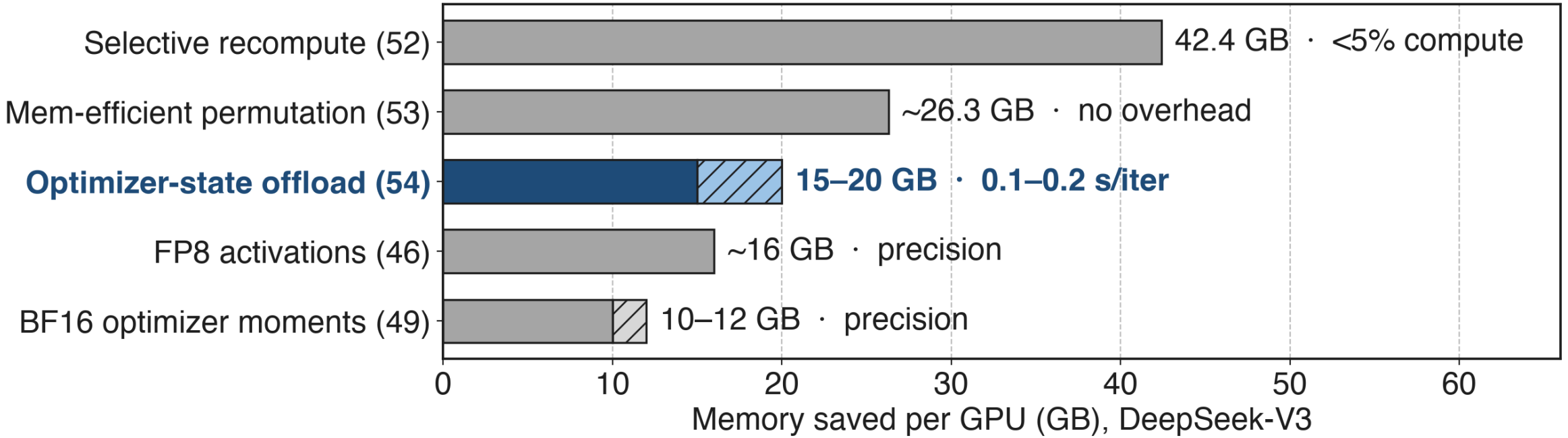


*Table 5. DeepSeek-V3: TP1 PP8 EP32 VPP4, MXFP8. *Qwen3-235B moves from TP2 / EP16 to TP1 / EP64 when offload is on*

Across configurations: 10–18% less memory for 1.6–2% throughput

Optimizer offload frees 15–20 GB per GPU



Each saving is reported separately against the 199.5 GB BF16 baseline (Table 3); they do not simply add up

Long context, DeepSeek-V3 on 256 H100: worst-case overhead about 2%

Kimi K2 and K3 offload the rest at trillion scale

- **Kimi K2** (1T, H800)
- Recompute, FP8, then offload the rest
- EP stays fully overlapped

- **Kimi K3** (2.8T)
- Offload or remote-offload per tensor
- Balances PP ranks' activations

Recompute what is cheap, compress what is safe, offload the rest

The cost: bandwidth, host memory, constraints

- **Copy bandwidth**: compute must outlast the copy
- PCIe traffic can touch EP (Kimi K2)
- Pinned host pool; last layer stays on GPU

- Needs TE; `NVTE_CPU_OFFLOAD_V1=1` on TE ≥ 2.10
- CUDA graphs: no `attn_norm` / `mlp_norm`
- State offload: no Megatron-FSDP

Summary

- Memory pins Qwen3-235B at TP2: **15%** throughput forgone
- **Insight**: copies hide under compute; 1F1B says when to move
- Module-level offload, layer-staggered reload, per-chunk handlers, state offload
- **–10.7%** memory for **–1.6%** speed; **+15%** via TP1; **15–20 GB** of optimizer state

https://arxiv.org/abs/2603.07685